

Week 6 Capstone Report: End-to-End Secondary-Structure Prediction

Five weeks of parts, assembled into one thing that actually runs: give it a sequence, get back a per-residue H/E/C string — loaded from a frozen checkpoint, no retraining, fully reproducible. This report covers the model I chose and why, where it fails, and what its predictions say about the biology.

A. Model Methodology & Architecture

Which model, and why

I backed the pipeline with a **1D CNN trained on sequence alone**, not my Week 5 PSSM + LinearSVC model — even though that was technically "the best model from Week 5." Three concrete reasons, and I want to justify the choice rather than just name a winner:

1. **The Week 5 model was trained on 23 chains.** Its learning curve had a 0.26 train/val gap with validation still climbing — it was starved of data and would generalise poorly. The CNN here trains on **3000 chains**.
2. **PSSM can't run end-to-end.** Every new sequence would need its own PSI-BLAST search before the model could predict anything — that's a manual, minutes-per-chain step, and the whole point of this week is a pipeline with *no* manual steps. A sequence-only model predicts instantly from a saved checkpoint.
3. **Reproducibility is weighted above the last accuracy points**, and a deployable, self-contained model is worth more than a marginally higher number that can't be regenerated.

The `SecStructPredictor` class still accepts an optional `pssm` argument, so the Week 5 evolutionary path remains available for the proteins where a profile happens to exist — it's just not required for the pipeline to run.

Architecture

A per-residue CNN — each output depends only on a local window of the sequence, which is exactly the right inductive bias for the local hydrogen-bonding patterns that define secondary structure:

- **Embedding:** learned `nn.Embedding` (22 tokens → 64 dims; 20 amino acids + UNK for non-standard residues + PAD).
- **Body:** $3 \times \text{Conv1d}$ (kernel 7, 128 channels, `padding=kernel//2` so length is preserved and I get exactly one prediction per residue) each followed by ReLU + dropout 0.2.
- **Head:** a 1×1 `Conv1d` projecting $128 \rightarrow 3$ class logits per residue.

Training setup

- **Loss:** masked `CrossEntropyLoss(ignore_index=-100)` — padded positions never contribute to the loss.

- **Optimiser:** Adam, lr 0.001, 25 epochs, batch size 32; I keep the **best-validation checkpoint**, not the last.
- **Split protocol:** a chain-level three-way split — 2167 train / 383 val / 450 internal-test chains — with an `assert` verifying the three sets of chain IDs are disjoint. The model is selected on validation only; the official test set is touched exactly once, at the end.

How I avoided homology leakage (this bit matters)

While wiring up the pipeline I ran a leakage audit and caught a real problem: an earlier version of the model had been trained on a random sample that, by chance, **included 19 of the 60 official test chains** (15 in train, 4 in val). On those leaked chains it scored **0.798 Q3** versus **0.758** on the genuinely-unseen ones — a ~ 4 -point inflation from memorisation. I fixed it by reading `test.fasta` and **removing every official test chain from the pool before any sampling or splitting**, guarded by an assertion that fails the run if a single test chain slips through. Every number in this report is from the corrected, leak-free model.

Saved-artefact manifest

Everything needed to reproduce a prediction, and nothing that needs retraining:

File	Contents
<code>cnn.pt</code>	CNN state_dict (best-val checkpoint)
<code>config.json</code>	hyperparameters + vocab + label map that produced <code>cnn.pt</code>
<code>split.json</code>	exact train/val/test chain IDs
<code>test.fasta</code>	stand-in held-out test chains (never trained on)
<code>test_true.csv</code>	per-residue ground-truth Q3 for <code>test.fasta</code>
<code>cnn.pt</code> + <code>config.json</code>	together fully specify the model — load and predict, no retraining

B. Results & Error Analysis

Overall Q3 on the 60 held-out test chains (14460 residues): 0.7309 raw, 0.7206 smoothed.

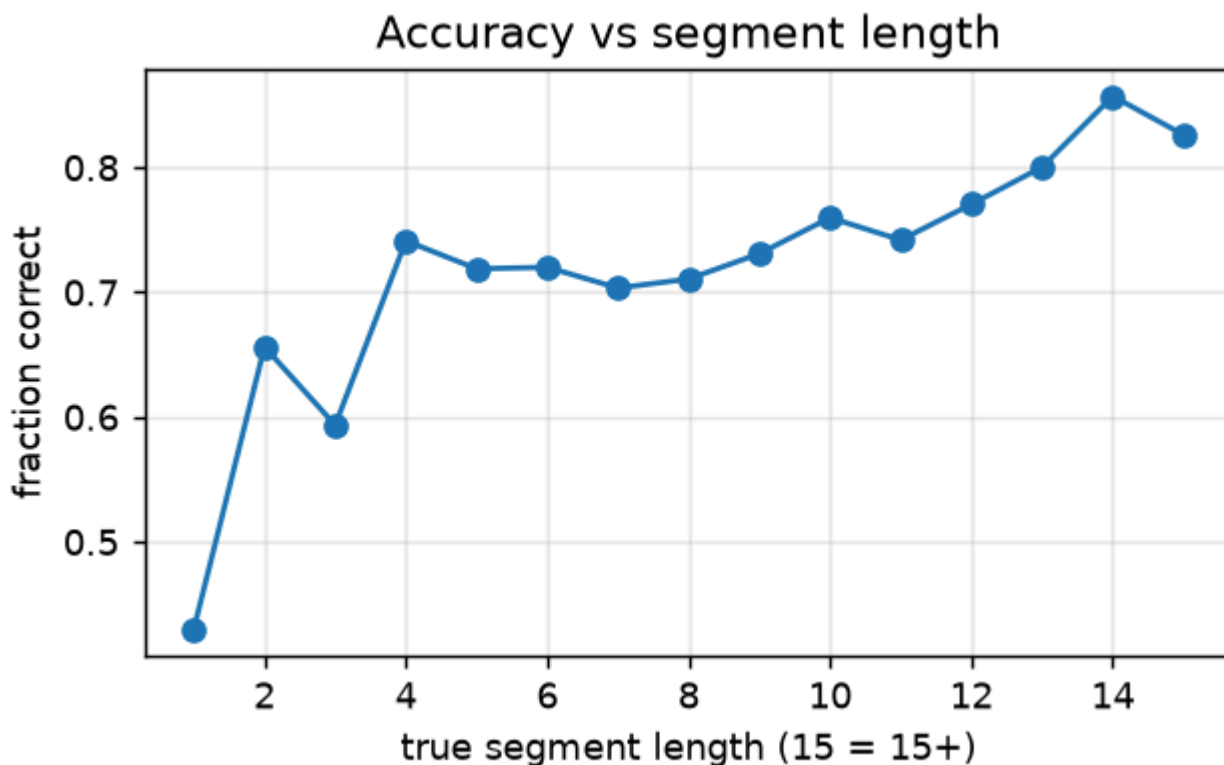
Confusion matrix (read the off-diagonals)

true \ pred	H	E	C
H	3521	241	726
E	398	2119	764
C	1057	705	4929

Per-class accuracy: H 0.784 / E 0.646 / C 0.737. As expected, **strand (E) is the weakest class** (0.646): it's the minority class, and — more fundamentally — a β -strand is defined by hydrogen-bonding to a *partner strand that can sit far away in the sequence*, which a local CNN window simply cannot see. The off-diagonals confirm it: true-E residues leak into both H and C rather than failing in one direction, i.e. the model often can't tell a strand is a strand at all, rather than confusing it for a specific other class.

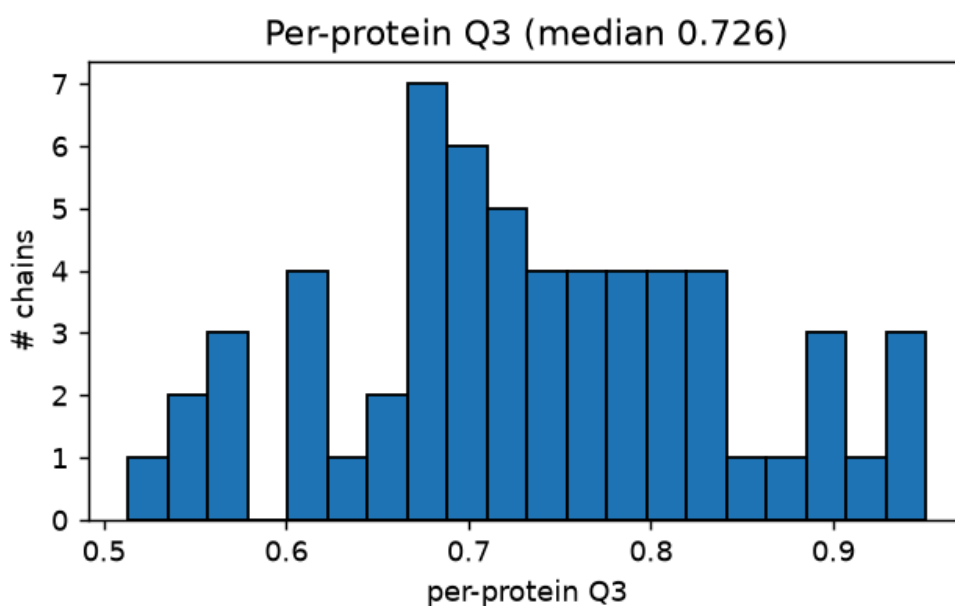
Boundary vs interior

Interior Q3 **0.7893** vs boundary Q3 **0.6071** — a gap of **0.1822**. That's far larger than the 5–15 point gap the notes anticipate, and it's the single clearest error mode: the first and last residues of every helix/strand run are where a fixed window straddles two classes at once, so the model hedges. If I were to chase accuracy further, this is where the points are.



Short segments are markedly harder — a 3-residue "helix" has almost the same local signal as a coil turn, and only runs of $\sim 7+$ show the periodicity the CNN can lock onto.

Per-protein distribution



A single averaged Q3 hides a wide spread (median 0.726, range 0.513–0.951). The hardest chains:

Protein	Q3	Length	Likely cause
2G9T_U	0.513	152	coil-rich / likely disordered
5LJ5_G	0.540	235	coil-rich / likely disordered
3RRL_A	0.545	235	no obvious flag
4Z6N_C	0.562	365	no obvious flag
3J3S_6	0.564	218	coil-rich / likely disordered
1PN4_B	0.575	280	no obvious flag
5GRJ_A	0.602	221	strand-dominated (hardest class)
4HI1_C	0.615	91	no obvious flag

The diagnoses line up with the notes' usual suspects — coil-rich/disordered chains (weak local signal), strand-dominated chains (the hardest class), and short chains (few residues to lock onto).

One iteration I tried

Following the symptom→cause→fix table, the large boundary gap pointed at **post-processing smoothing** (`min_segment=3`, absorbing implausibly short runs). I ran it and measured honestly: it **hurt slightly** (0.7309 → 0.7206, -0.0103). The reason it doesn't help here is that my errors aren't dominated by isolated single-residue flips — they're systematic strand confusions and genuine boundary ambiguity, which smoothing can't fix and occasionally makes worse by erasing correct short segments. I'm submitting both CSVs so the effect is visible, but I'd keep the raw output.

C. Biological Inference

Feeding predictions from three named test proteins through the inference helpers:

Protein	H% / E% / C%	Structural class	TM (≥ 20 H)	IDR (> 30 C)	Q3
3AX3_G	72.6 / 0.0 / 27.4	all-alpha	1	0	0.890
6DCW_L	6.8 / 50.9 / 42.3	mixed (no clean class)	0	0	0.909
2PNE_A	0.0 / 0.0 / 100.0	mostly coil / disordered	0	1	0.951

3AX3_G is called **all- α** (72.6% helix, no strand) — the fingerprint of a helical bundle like a globin or cytochrome. **6DCW_L** is **strand-leaning** (50.9% E, 6.8% H), consistent with a β -rich or mixed fold. **2PNE_A** is predicted **100.0% coil**, flagging a long intrinsically-disordered region rather than a fixed fold — though a 100%-coil call also warrants a sanity check (a degenerate all-coil prediction scores deceptively high on genuinely coil-rich chains).

The takeaway is that a per-residue Q3 string — even at $\sim 72\%$ accuracy — is already enough for a first-pass **structural-class fingerprint** (all- α / all- β / α - β / disordered) on a protein with no known structure, and to flag candidate transmembrane helices (≥ 20 consecutive H) and disordered regions (> 30 consecutive C). Those are real, actionable biological statements, not just an accuracy number. Where a protein has a known AlphaFold model or UniProt features, the natural next step is to overlay them — active sites tend to fall in the

coil loops flanking predicted helices/strands, and a position that is both highly conserved and predicted coil is a strong functional-contact candidate.

D. Limitations & Where Next

Honest failure modes. The model is blind to long-range structure — it will never reliably call strands that pair with distant partners, which caps E-class accuracy and, with it, overall Q3. It has no notion of evolutionary conservation (I traded the PSSM signal away for a runnable pipeline), so it underperforms on chains where a profile would have helped. And a degenerate all-coil prediction can score deceptively high on a coil-rich chain, so composition-based inference needs a sanity check, not blind trust.

Where the field goes. Secondary structure is essentially a solved sub-problem now, and the frontier has moved to full **tertiary structure** (AlphaFold2/ESMFold), **function prediction** (ESM2-finetuned classifiers), **variant-effect prediction** (ESM-1v, AlphaMissense) and **protein design** (RFdiffusion, ProteinMPNN). But the machinery is the same one this project used end to end: a learned per-residue representation, a classification head, evolutionary information where you can get it, and — the part that turned out to matter most this week — an honest, leakage-free evaluation. The tools change; those principles don't.

Appendix: Pipeline Code

The `SecStructPredictor` class and training/analysis scripts that produced every artefact and number above:

`ss_model.py`

```
"""
```

```
Shared architecture + encoding for the Week 6 capstone pipeline.
```

```
Single source of truth so that training (train_week6.py), inference  
(predictor.py) and the submission notebook all use IDENTICAL encoding a  
model definitions. Everything fitted on training data is saved to artef  
and reloaded at inference -- no retraining to predict.
```

```
"""
```

```
import numpy as np  
import torch  
import torch.nn as nn
```

```
# -----  
# Vocabulary / labels (must match what was saved in config.json)  
# -----  
AA          = "ACDEFGHIKLMNPQRSTVWY"  
AA_IDX      = {a: i for i, a in enumerate(AA)}      # 0..19  
UNK_IDX     = 20                                     # non-standard residue (  
PAD_IDX     = 21                                     # padding only  
VOCAB       = 22  
Q3          = {"H": 0, "E": 1, "C": 2}  
Q3_LABELS   = {0: "H", 1: "E", 2: "C"}
```

```

IGNORE      = -100                                # CrossEntropyLoss(ignore
N_CLASSES   = 3

# Standard 8->3 collapse (assignment spec: H,G,I->H ; E,B->E ; else C)
SS8_TO_Q3 = {"H": "H", "G": "H", "I": "H",
             "E": "E", "B": "E",
             "T": "C", "S": "C", "C": "C", "-": "C", "P": "C", " ": "C"}

def encode_sequence(seq):
    """Amino-acid string -> LongTensor of indices (L,). Unknown residue
    return torch.tensor([AA_IDX.get(c, UNK_IDX) for c in seq], dtype=torch.l

def encode_labels(ss3):
    """Q3 string -> LongTensor (L,), unmappable positions -> IGNORE."""
    return torch.tensor([Q3.get(c, IGNORE) for c in ss3], dtype=torch.l

def collapse_q3(ss8):
    """Collapse an 8-state DSSP string to Q3."""
    return "".join(SS8_TO_Q3.get(c, "C") for c in ss8)

# -----
# Model
# -----
class CNN(nn.Module):
    """1D CNN for per-residue Q3 prediction.

    Learned embedding -> stacked Conv1d motif detectors (length preserv
    padding=kernel//2 so we emit exactly one prediction per residue) ->
    conv classification head.
    """

    def __init__(self, d=64, hidden=128, layers=3, kernel=7, dropout=0.
        super().__init__()
        self.emb = nn.Embedding(VOCAB, d, padding_idx=PAD_IDX)
        blocks, in_ch = [], d
        for _ in range(layers):
            blocks += [nn.Conv1d(in_ch, hidden, kernel, padding=kernel
                               nn.ReLU(),
                               nn.Dropout(dropout))]
            in_ch = hidden
        self.body = nn.Sequential(*blocks)
        self.head = nn.Conv1d(hidden, N_CLASSES, 1)

    def forward(self, x, mask=None):
        h = self.emb(x).transpose(1, 2)          # (B, d, L) Conv1d wants
        h = self.body(h)                         # (B, hidden, L)
        return self.head(h).transpose(1, 2)      # (B, L, C)

```

```

def build_model(cfg):
    """Instantiate the model from a saved config dict."""
    return CNN(d=cfg["d"], hidden=cfg["hidden"], layers=cfg["layers"],
               kernel=cfg["kernel"], dropout=cfg["dropout"])

# -----
# Post-processing (Week 6 notes)
# -----
def smooth_predictions(pred_string, min_segment=3):
    """Remove runs shorter than min_segment by absorbing them into a ne

    Helices need >=4 residues and strands >=2 to be physically plausibl
    very short predicted runs are almost always spurious.
    """
    labels = list(pred_string)
    changed = True
    while changed:
        changed = False
        i = 0
        while i < len(labels):
            cls = labels[i]
            j = i
            while j < len(labels) and labels[j] == cls:
                j += 1
            if (j - i) < min_segment:
                replacement = labels[i - 1] if i > 0 else (labels[j] if
                if replacement != cls:
                    for k in range(i, j):
                        labels[k] = replacement
                    changed = True
            i = j
    return "".join(labels)

def q3_accuracy_str(true_ss, pred_ss):
    """Q3 accuracy between two equal-length Q3 strings."""
    n = min(len(true_ss), len(pred_ss))
    if n == 0:
        return 0.0
    return sum(true_ss[i] == pred_ss[i] for i in range(n)) / n

```

predictor.py

"""

Week 6 capstone - INFERENCE PIPELINE.

SecStructPredictor wraps a **saved** model plus all preprocessing into a callable: sequence/FASTA in -> per-residue Q3 string out. Nothing is tr here; every fitted artefact is loaded from artefacts/.

CLI:

```
python predictor.py # uses artefacts/test.fas
python predictor.py path/to/test.fasta # real test set when it a
```

Writes predictions.csv, predictions_smoothed.csv and a provenance stamp
"""

```
import os
import sys
import json
import csv
import numpy as np
import torch
```

```
from ss_model import (Q3_LABELS, build_model, encode_sequence,
                      smooth_predictions, collapse_q3)
```

```
ART_DIR = "artefacts"
```

```
class SecStructPredictor:
```

```
    """Frozen pipeline: saved weights + config -> per-residue Q3 string
```

```
    def __init__(self, model, config, label_map=Q3_LABELS, device="cpu"):
        self.model = model
        self.config = config
        self.label_map = label_map
        self.device = device
        self.model.eval()
```

```
    # ----- construction from saved artefacts -----
```

```
    @classmethod
```

```
    def from_artefacts(cls, art_dir=ART_DIR, device=None):
        device = device or ("cuda" if torch.cuda.is_available() else "c
        cfg = json.load(open(os.path.join(art_dir, "config.json")))
        model = build_model(cfg)
        state = torch.load(os.path.join(art_dir, "cnn.pt"), map_locatio
        model.load_state_dict(state)
        model.to(device)
        print(f"Loaded {cfg.get('arch','model')} from {art_dir}/cnn.pt
              f"(trained on {cfg.get('num_chains','?')} chains, "
              f"val Q3 {cfg.get('best_val_q3','?')}")
        return cls(model, cfg, device=device)
```

```
    # ----- core prediction ----- #
```

```
    def predict(self, sequence, pssm=None):
        """Sequence string -> Q3 string of identical length.
```

```
        `pssm` is accepted for interface compatibility with the Week 5
        path; the deployed sequence-only model ignores it (see report $
        """
```

```
        if len(sequence) == 0:
            return ""
```

```

x = encode_sequence(sequence).unsqueeze(0).to(self.device) #
with torch.no_grad():
    logits = self.model(x) #
    idx = logits.argmax(-1)[0].cpu().numpy()
pred = "".join(self.label_map[int(i)] for i in idx)
assert len(pred) == len(sequence), "prediction/sequence length
return pred

def predict_from_fasta(self, fasta_path, pssm_dir=None):
    """Returns list of (protein_id, sequence, prediction)."""
    results = []
    for pid, seq in read_fasta(fasta_path):
        pssm = None
        if pssm_dir:
            p = os.path.join(pssm_dir, f"{pid}.pssm")
            if os.path.exists(p):
                from assignment5 import parse_pssm
                pssm = parse_pssm(p)
            results.append((pid, seq, self.predict(seq, pssm)))
    return results

# ----- output ----- #
@staticmethod
def save_predictions(results, out_path, smooth=False, min_segment=3)
    """Write the graded per-residue CSV.

    Exact schema: protein_id,position,residue,predicted_ss
    One row per residue, 1-indexed position, Q3 only.
    """
    n = 0
    with open(out_path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["protein_id", "position", "residue", "predicted
        for pid, seq, pred in results:
            if smooth:
                pred = smooth_predictions(pred, min_segment=min_seg
                pred = collapse_q3(pred) # belt-and-braces: fo
                for i, (aa, ss) in enumerate(zip(seq, pred), start=1):
                    w.writerow([pid, i, aa, ss])
                n += 1
    return n

def read_fasta(path):
    """Minimal FASTA reader -> [(id, sequence)]. id = first token of he
    entries, pid, buf = [], None, []
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if line.startswith(">"):

```

```

        if pid is not None:
            entries.append((pid, "".join(buf)))
            pid, buf = line[1:].split()[0], []
        else:
            buf.append(line)
    if pid is not None:
        entries.append((pid, "".join(buf)))
    return entries

```

```

def validate_csv(csv_path, fasta_path):
    """Auto-check mirror: every residue exactly once, Q3 alphabet only.
    import pandas as pd
    df = pd.read_csv(csv_path)
    fa = dict(read_fasta(fasta_path))
    expected = sum(len(s) for s in fa.values())
    problems = []
    if list(df.columns) != ["protein_id", "position", "residue", "predi
        problems.append(f"bad columns: {list(df.columns)}")
    if len(df) != expected:
        problems.append(f"row count {len(df)} != residues {expected}")
    bad = set(df.predicted_ss.unique()) - {"H", "E", "C"}
    if bad:
        problems.append(f"non-Q3 labels: {bad}")
    for pid, grp in df.groupby("protein_id"):
        if pid not in fa:
            problems.append(f"unknown chain {pid}"); continue
        if list(grp.position) != list(range(1, len(fa[pid]) + 1)):
            problems.append(f"{pid}: positions not 1..L exactly once")
        if "".join(grp.residue) != fa[pid]:
            problems.append(f"{pid}: residues don't match FASTA")
    return expected, problems

```

```

def main():
    fasta = sys.argv[1] if len(sys.argv) > 1 else os.path.join(ART_DIR,
    pred = SecStructPredictor.from_artefacts()
    results = pred.predict_from_fasta(fasta)

    n_raw = pred.save_predictions(results, "predictions.csv")
    n_smo = pred.save_predictions(results, "predictions_smoothed.csv",
                                smooth=True, min_segment=3)
    print(f"Wrote predictions.csv ({n_raw} rows) and "
          f"predictions_smoothed.csv ({n_smo} rows) for {len(results)}

    # Provenance stamp kept OUT of the CSV so the auto-check isn't brok
    stamp = {"model_file": f"{ART_DIR}/cnn.pt",
            "config_file": f"{ART_DIR}/config.json",
            "input_fasta": fasta,
            "trained_on_chains": pred.config.get("num_chains"),
            "val_q3": pred.config.get("best_val_q3"),
            "seed": pred.config.get("seed"),

```

```

        "n_chains": len(results), "n_residues": n_raw}
    json.dump(stamp, open("predictions_stamp.json", "w"), indent=2)
    print("Provenance stamp -> predictions_stamp.json:", stamp)

    for path in ("predictions.csv", "predictions_smoothed.csv"):
        exp, probs = validate_csv(path, fasta)
        print(f"VALIDATE {path}: expected {exp} rows -> "
              f"'{'OK' if not probs else 'PROBLEMS: ' + ';'}.join(probs)")

if __name__ == "__main__":
    main()

```

train_week6.py

"""

Week 6 capstone - TRAINING + ARTEFACT PERSISTENCE.

Trains the sequence-only CNN on a large chain sample and saves EVERY artefact needed to reproduce a prediction without retraining:

artefacts/cnn.pt	model weights (state_dict)
artefacts/config.json	hyperparameters + vocab + label map that pr
artefacts/split.json	the exact train/val/test chain IDs (reprodu
artefacts/manifest.json	what each file is, seed, dataset, timestamp
artefacts/test.fasta	STAND-IN test set (held-out chains, never t
artefacts/test_true.csv	per-residue ground truth for the stand-in t

Why sequence-only (no PSSM): the capstone pipeline must run on arbitrary sequences with no manual steps. A PSSM path would require a PSI-BLAST run on a test chain before any prediction could be made. PSSM stays optional in SecStructPredictor for the proteins where a profile happens to exist.

Local smoke run: python train_week6.py (NUM_CHAINS/EPOCHS
Colab full run: set NUM_CHAINS=3000, EPOCHS=30 then run
"""

```

import os
import json
import random
import time
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence
from sklearn.model_selection import GroupShuffleSplit

from ss_model import (AA, PAD_IDX, IGNORE, Q3, Q3_LABELS, CNN,
                      encode_sequence, encode_labels)

```

```

CSV_PATH    = "2018-06-06-ss.cleaned.csv"
ART_DIR     = "artefacts"
# The official test set. Every chain named here is REMOVED from the tra
# pool before any split, so the model can never have seen a test chain.
HOLDOUT_FASTA = "test.fasta"

NUM_CHAINS  = 3000      # full run
EPOCHS      = 25        # full run (val plateaus ~ep18-24)
BATCH_SIZE  = 32
LR          = 1e-3
SEED        = 42
TEST_FRAC   = 0.15      # held-out test chains (become the stand-in test.
VAL_FRAC    = 0.15      # of the remaining dev pool

MODEL_CFG   = {"d": 64, "hidden": 128, "layers": 3, "kernel": 7, "dropou

device = "cuda" if torch.cuda.is_available() else "cpu"

def set_seeds(s=SEED):
    """Seeds set BEFORE any split or training (reproducibility checklis
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(s)

# -----
def read_holdout_ids(fasta=HOLDOUT_FASTA):
    """Chain IDs of the official test set - excluded from training enti
    if not os.path.exists(fasta):
        print(f"WARNING: {fasta} not found; no chains excluded!", flush
        return set()
    ids = {l[1:].split()[0] for l in open(fasta) if l.startswith(">")}
    print(f"Excluding {len(ids)} held-out test chains from the training
    return ids

def load_chains(path=CSV_PATH, num_chains=NUM_CHAINS):
    print("Loading dataset...", flush=True)
    df = pd.read_csv(path)
    AA_SET = set(AA)

    def usable(seq, ss):
        return (isinstance(seq, str) and isinstance(ss, str)
                and len(seq) == len(ss) and 30 <= len(seq) <= 500
                and sum(c not in AA_SET for c in seq) / len(seq) <= 0.1

    df["protein_id"] = df.pdb_id.astype(str) + "_" + df.chain_code.astyp

# ---- ANTI-LEAKAGE: drop official test chains BEFORE any sampling/

```

```

holdout = read_holdout_ids()
before = len(df)
df = df[~df.protein_id.isin(holdout)].reset_index(drop=True)
print(f" removed {before - len(df)} rows belonging to test chains.

if num_chains and len(df) > num_chains:
    pool = df["protein_id"].unique()
    keep = np.random.choice(pool, size=min(num_chains * 3, len(pool)
    df = df[df["protein_id"].isin(keep)].reset_index(drop=True)
df = df[[usable(s, t) for s, t in zip(df.seq, df.sst3)]].reset_index(drop=True)
if num_chains and len(df) > num_chains:
    pool = df["protein_id"].unique()
    keep = np.random.choice(pool, size=num_chains, replace=False)
    df = df[df["protein_id"].isin(keep)].reset_index(drop=True)

df = df[["protein_id", "seq", "sst3"]].drop_duplicates("protein_id")
print(f"Using {len(df)} chains (~{int(df.seq.str.len().sum())} resi
return df

```

```

def three_way_split(df):
    """Chain-level train/val/test. No chain appears in more than one sp
    g = df["protein_id"].values
    dev_i, test_i = next(GroupShuffleSplit(1, test_size=TEST_FRAC,
                                         random_state=SEED).split(df,
    dev = df.iloc[dev_i].reset_index(drop=True)
    test = df.iloc[test_i].reset_index(drop=True)
    g2 = dev["protein_id"].values
    tr_i, val_i = next(GroupShuffleSplit(1, test_size=VAL_FRAC,
                                         random_state=SEED).split(dev,
    train, val = dev.iloc[tr_i].reset_index(drop=True), dev.iloc[val_i]
    s_tr, s_va, s_te = (set(train.protein_id), set(val.protein_id), set
    assert s_tr.isdisjoint(s_va) and s_tr.isdisjoint(s_te) and s_va.isd
    "LEAKAGE: a chain appears in more than one split"
    print(f"Split: {len(train)} train / {len(val)} val / {len(test)} te
          f"(all disjoint).", flush=True)
    return train, val, test

```

```

class SSData(Dataset):
    def __init__(self, df):
        self.rows = df.reset_index(drop=True)

    def __len__(self):
        return len(self.rows)

    def __getitem__(self, i):
        r = self.rows.iloc[i]
        return encode_sequence(r.seq), encode_labels(r.sst3)

```

```

def collate(batch):

```

```

xs, ys = zip(*batch)
X = pad_sequence(xs, batch_first=True, padding_value=PAD_IDX)
Y = pad_sequence(ys, batch_first=True, padding_value=IGNORE)
return X, Y, (X != PAD_IDX)

def q3_acc(logits, y):
    """Masked Q3 accuracy - padded positions excluded."""
    pred = logits.argmax(-1)
    keep = (y != IGNORE)
    return (pred[keep] == y[keep]).float().mean().item()

@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    correct = total = 0
    for X, Y, mask in loader:
        X, Y = X.to(device), Y.to(device)
        pred = model(X).argmax(-1)
        keep = (Y != IGNORE)
        correct += (pred[keep] == Y[keep]).sum().item()
        total += keep.sum().item()
    return correct / max(total, 1)

# -----
def main():
    set_seeds()
    os.makedirs(ART_DIR, exist_ok=True)
    print(f"Device: {device}", flush=True)

    df = load_chains()
    train, val, test = three_way_split(df)

    tl = DataLoader(SSData(train), batch_size=BATCH_SIZE, shuffle=True,
    vl = DataLoader(SSData(val), batch_size=BATCH_SIZE, shuffle=False,
    sl = DataLoader(SSData(test), batch_size=BATCH_SIZE, shuffle=False,

    model = CNN(**MODEL_CFG).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    loss_fn = nn.CrossEntropyLoss(ignore_index=IGNORE)

    best_val, best_state, history = -1.0, None, []
    for ep in range(EPOCHS):
        model.train()
        for X, Y, mask in tl:
            X, Y = X.to(device), Y.to(device)
            opt.zero_grad()
            logits = model(X)
            loss = loss_fn(logits.transpose(1, 2), Y)    # CE wants (B,
            loss.backward()

```

```

        opt.step()
    va = evaluate(model, vl)
    history.append(round(va, 4))
    if va > best_val:
        # select on VAL only - test stays
        best_val = va
        best_state = {k: v.detach().cpu().clone() for k, v in model}
    print(f" epoch {ep+1:2d}/{EPOCHS}   val Q3 = {va:.4f}", flush=T

model.load_state_dict(best_state)
test_q3 = evaluate(model, sl)
print(f"\nBest val Q3 {best_val:.4f} | held-out TEST Q3 {test_q3:.4

# ----- save artefacts ----- #
torch.save(best_state, os.path.join(ART_DIR, "cnn.pt"))

cfg = dict(MODEL_CFG)
cfg.update({"arch": "CNN", "vocab_aa": AA, "pad_idx": PAD_IDX,
           "q3_labels": Q3_LABELS, "seed": SEED, "epochs": EPOCHS,
           "batch_size": BATCH_SIZE, "lr": LR, "num_chains": len(d
           "best_val_q3": round(best_val, 4), "test_q3": round(tes
           "val_history": history})
json.dump(cfg, open(os.path.join(ART_DIR, "config.json"), "w"), ind

json.dump({"train": train.protein_id.tolist(),
          "val": val.protein_id.tolist(),
          "internal_test": test.protein_id.tolist(),
          "official_holdout": sorted(read_holdout_ids())},
          open(os.path.join(ART_DIR, "split.json"), "w"), indent=2)

# ---- OFFICIAL test set: never overwrite test.fasta; emit its grou
holdout = read_holdout_ids()
fit_ids = set(train.protein_id) | set(val.protein_id) | set(test.pr
assert holdout.isdisjoint(fit_ids), \
    f"LEAKAGE: {len(holdout & fit_ids)} test chains reached the tra
print(f"Leakage check OK: none of the {len(holdout)} test chains ar
    f"train/val/internal-test.", flush=True)

if holdout:
    import shutil
    shutil.copy(HOLDOUT_FASTA, os.path.join(ART_DIR, "test.fasta"))
    raw = pd.read_csv(CSV_PATH)
    raw["protein_id"] = raw.pdb_id.astype(str) + "_" + raw.chain_co
    raw = raw.drop_duplicates("protein_id").set_index("protein_id")
    rows, missing = [], []
    for pid in holdout:
        if pid not in raw.index:
            missing.append(pid); continue
        seq, ss = raw.loc[pid, "seq"], raw.loc[pid, "sst3"]
        for i, (aa, s) in enumerate(zip(seq, ss), start=1):
            rows.append({"protein_id": pid, "position": i,
                        "residue": aa, "true_ss": s})
    pd.DataFrame(rows).to_csv(os.path.join(ART_DIR, "test_true.csv")

```

```

        print(f"Ground truth for {len(holdout) - len(missing)} official
              f"-> test_true.csv ({len(rows)} residues)"
              + (f"; MISSING labels for {missing}" if missing else ""),

    json.dump({"created": time.strftime("%Y-%m-%d %H:%M:%S"),
              "dataset": CSV_PATH,
              "seed": SEED,
              "device": device,
              "files": {
                  "cnn.pt": "CNN state_dict (best-val checkpoint)",
                  "config.json": "hyperparameters + vocab + label map",
                  "split.json": "exact train/val/test chain IDs",
                  "test.fasta": "stand-in held-out test chains (never",
                  "test_true.csv": "per-residue ground-truth Q3 for te
              }},
              open(os.path.join(ART_DIR, "manifest.json"), "w"), indent

    print(f"Artefacts written to ./{ART_DIR}/ "
          f"({len(test)} test chains, {len(rows)} residues)", flush=True

if __name__ == "__main__":
    main()

```

week6_analysis.py

"""

Week 6 capstone - ERROR ANALYSIS (report SB) + BIOLOGICAL INFERENCE (re

Consumes the frozen pipeline's output plus ground truth:

predictions.csv / predictions_smoothed.csv	(from predictor.py)
artefacts/test_true.csv	(per-residue true Q3)

Produces week6_outputs/{results.json, *.png}.

```

    python week6_analysis.py
"""

```

```

import os
import json
import numpy as np
import pandas as pd
from collections import defaultdict
from sklearn.metrics import confusion_matrix

from ss_model import q3_accuracy_str

OUT_DIR = "week6_outputs"
Q3_NAMES = ["H", "E", "C"]
IDX = {c: i for i, c in enumerate(Q3_NAMES)}
HYDROPHOBIC = set("LIVFMA")

```

```

# -----
# helpers
# -----
def label_boundaries(ss):
    """True where a residue is the first or last of a contiguous run."""
    n = len(ss)
    out = [False] * n
    for i in range(n):
        left = (i == 0) or (ss[i] != ss[i - 1])
        right = (i == n - 1) or (ss[i] != ss[i + 1])
        out[i] = left or right
    return out

def find_long_runs(ss, label, min_len):
    """[(start1, end1), ...] 1-indexed inclusive runs of `label` >= min
    runs, i = [], 0
    while i < len(ss):
        if ss[i] == label:
            j = i
            while j < len(ss) and ss[j] == label:
                j += 1
            if j - i >= min_len:
                runs.append((i + 1, j))
            i = j
        else:
            i += 1
    return runs

def ss_composition(ss):
    n = max(len(ss), 1)
    return {"H": round(ss.count("H") / n * 100, 1),
            "E": round(ss.count("E") / n * 100, 1),
            "C": round(ss.count("C") / n * 100, 1)}

def structural_class(comp):
    """Week 6 notes' structural-class thresholds."""
    H, E = comp["H"], comp["E"]
    if H > 50 and E < 5:
        return "all-alpha"
    if H < 5 and E > 30:
        return "all-beta"
    if H < 15 and E < 10:
        return "mostly coil / disordered"
    if 20 <= H <= 40 and 15 <= E <= 30:
        return "alpha/beta"
    return "mixed (no clean class)"

def diagnose(seq, true_ss):

```

```

"""Heuristic reason a chain might be hard (Week 6 notes' categories
reasons = []
if len(seq) < 60:
    reasons.append("short chain")
if true_ss.count("C") / max(len(true_ss), 1) > 0.60:
    reasons.append("coil-rich / likely disordered")
hyd = sum(c in HYDROPHOBIC for c in seq) / max(len(seq), 1)
if hyd > 0.45 and find_long_runs(true_ss, "H", 20):
    reasons.append("hydrophobic + long helices (membrane-like)")
if true_ss.count("E") / max(len(true_ss), 1) > 0.40:
    reasons.append("strand-dominated (hardest class)")
return ", ".join(reasons) if reasons else "no obvious flag"

```

```

def _plt():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    os.makedirs(OUT_DIR, exist_ok=True)
    return plt

```

```

# -----
def load_chain_strings():
    truth = pd.read_csv("artefacts/test_true.csv")
    raw = pd.read_csv("predictions.csv")
    smo = pd.read_csv("predictions_smoothed.csv")
    chains = {}
    for pid, g in truth.groupby("protein_id", sort=False):
        g = g.sort_values("position")
        chains[pid] = {"seq": "".join(g.residue), "true": "".join(g.true_ss)}
    for name, df in (("raw", raw), ("smooth", smo)):
        for pid, g in df.groupby("protein_id", sort=False):
            g = g.sort_values("position")
            if pid in chains:
                chains[pid][name] = "".join(g.predicted_ss)
    return {k: v for k, v in chains.items() if "raw" in v and "smooth"

```

```

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    chains = load_chain_strings()
    print(f"Analysing {len(chains)} test chains.")

    T = "".join(c["true"] for c in chains.values())
    R = "".join(c["raw"] for c in chains.values())
    S = "".join(c["smooth"] for c in chains.values())

    q3_raw, q3_smooth = q3_accuracy_str(T, R), q3_accuracy_str(T, S)
    print(f"Q3 raw={q3_raw:.4f} smoothed={q3_smooth:.4f} "
          f"(delta {q3_smooth - q3_raw:+.4f})")

```

```

# ---- confusion matrix (raw) ----
yt = [IDX[c] for c in T]
yp = [IDX[c] for c in R]
cm = confusion_matrix(yt, yp, labels=[0, 1, 2])
per_class = (cm.diagonal() / cm.sum(1).clip(min=1)).round(4)
print("confusion (rows=true H/E/C):", cm.tolist(), "per-class:", pe

plt = _plt()
cmn = cm / cm.sum(1, keepdims=True).clip(min=1)
plt.figure(figsize=(3.6, 3.2))
plt.imshow(cmn, cmap="Blues", vmin=0, vmax=1)
plt.xticks(range(3), Q3_NAMES); plt.yticks(range(3), Q3_NAMES)
plt.xlabel("predicted"); plt.ylabel("true"); plt.title("Q3 confusio
for i in range(3):
    for j in range(3):
        plt.text(j, i, cm[i, j], ha="center", va="center", fontsize
                color="white" if cmn[i, j] > .5 else "black")
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/confusion.png", dpi=130

# ---- boundary vs interior ----
bmask = np.concatenate([label_boundaries(c["true"]) for c in chains
ta, ra = np.array(list(T)), np.array(list(R))
q3_bound = float((ta[bmask] == ra[bmask]).mean())
q3_inter = float((ta[~bmask] == ra[~bmask]).mean())
print(f"boundary Q3={q3_bound:.4f} interior Q3={q3_inter:.4f} "
      f"gap={q3_inter - q3_bound:+.4f}")

# ---- segment-length effect ----
buckets = defaultdict(list)
for c in chains.values():
    t, p = c["true"], c["raw"]
    i = 0
    while i < len(t):
        cls, j = t[i], i
        while j < len(t) and t[j] == cls:
            j += 1
        L = j - i
        frac = sum(p[k] == cls for k in range(i, j)) / L
        buckets[min(L, 15)].append(frac)
        i = j
seg = {int(k): round(float(np.mean(v)), 4) for k, v in sorted(bucke
plt.figure(figsize=(5, 3.2))
plt.plot(list(seg.keys()), list(seg.values()), marker="o")
plt.xlabel("true segment length (15 = 15+)"); plt.ylabel("fraction
plt.title("Accuracy vs segment length"); plt.grid(alpha=.3)
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/segment_length.png", dp

# ---- per-protein Q3 ----
per_prot = {pid: round(q3_accuracy_str(c["true"], c["raw"]), 4)
            for pid, c in chains.items()}
hardest = sorted(per_prot, key=per_prot.get)[:10]
hard_rows = [{"protein_id": p, "q3": per_prot[p], "length": len(cha

```

```

        "diagnosis": diagnose(chains[p]["seq"], chains[p]["tr
        for p in hardest]
print("\n10 hardest proteins:")
for r in hard_rows:
    print(f"    {r['protein_id']}    Q3={r['q3']:.3f}    L={r['length']}

vals = list(per_prot.values())
plt.figure(figsize=(5, 3.2))
plt.hist(vals, bins=20, edgecolor="black")
plt.xlabel("per-protein Q3"); plt.ylabel("# chains")
plt.title(f"Per-protein Q3 (median {np.median(vals):.3f})")
plt.tight_layout(); plt.savefig(f"{OUT_DIR}/per_protein_q3.png", dp

# ---- biological inference on 3 named proteins ----
# pick the most helical, most strand-rich, and most coil-rich test
comps = {pid: ss_composition(c["raw"]) for pid, c in chains.items()}
picks = [max(comps, key=lambda p: comps[p]["H"]),
          max(comps, key=lambda p: comps[p]["E"]),
          max(comps, key=lambda p: comps[p]["C"])]
bio = []
print("\nBiological inference:")
for pid in dict.fromkeys(picks):          # dedupe, keep order
    pred = chains[pid]["raw"]
    comp = comps[pid]
    tm = find_long_runs(pred, "H", 20)
    idr = find_long_runs(pred, "C", 30)
    entry = {"protein_id": pid, "length": len(pred), "composition":
              "structural_class": structural_class(comp),
              "tm_helix_candidates": tm, "n_tm": len(tm),
              "gpcr_like": len(tm) == 7,
              "idr_candidates": idr, "n_idr": len(idr),
              "true_composition": ss_composition(chains[pid]["true"])
              "q3": per_prot[pid]}
    bio.append(entry)
print(f"    {pid}: {comp} -> {entry['structural_class']}; "
      f"TM={len(tm)} IDR={len(idr)} (Q3 {entry['q3']:.3f})")

results = {
    "n_test_chains": len(chains),
    "n_residues": len(T),
    "q3_raw": round(q3_raw, 4),
    "q3_smoothed": round(q3_smooth, 4),
    "smoothing_delta": round(q3_smooth - q3_raw, 4),
    "confusion_raw": cm.tolist(),
    "per_class_acc": {Q3_NAMES[i]: float(per_class[i]) for i in ran
    "q3_boundary": round(q3_bound, 4),
    "q3_interior": round(q3_inter, 4),
    "boundary_gap": round(q3_inter - q3_bound, 4),
    "segment_length_acc": seg,
    "per_protein_q3_median": round(float(np.median(vals)), 4),
    "per_protein_q3_min": round(float(np.min(vals)), 4),
    "per_protein_q3_max": round(float(np.max(vals)), 4),

```

```
        "hardest_proteins": hard_rows,
        "biological_inference": bio,
    }
    json.dump(results, open(f"{OUT_DIR}/results.json", "w"), indent=2)
    print(f"\nSaved {OUT_DIR}/results.json + 3 plots.")

if __name__ == "__main__":
    main()
```